

When the Product Won't Collapse: A Negative Result on int8 Tensor-Core MMQ

Hanzo AI — ML Systems

hanzo-kernel Paper 07.5 — part of the *One Source, Every Backend* series

Abstract

A single-source kernel DSL collapses the $\text{backends} \times \text{ops} \times \text{types}$ product for the decode matvecs, norms, RoPE, attention, and the whole elementwise fabric of a model. This paper reports, in full, the one operation that resists: block-quantized integer matrix–matrix multiply on the tensor cores (MMQ), the kernel that governs prefill throughput. The result is negative and structural, and we report it as such because it is more useful than a success story. Through the portable high-level WMMA (cooperative-matrix) API, MMQ is capped at roughly one-tenth of the hand-tuned kernel — not for want of tuning, but because the API's accumulator fragment is *opaque*, forcing the per-32-block quantization scale to be applied via a store→synchronize→reload round-trip through shared memory once per block. The proof that this serialization is the wall, not occupancy, is that a *better-tiled* version is *slower*. The higher-ceiling route — manual `mma.sync` with an exposed fragment-to-lane map, so the scale is applied in register — is viable but per-backend by nature (the DSL's int8 matrix-core intrinsic is CUDA-only), so it does not collapse the product; it writes one more backend-specific kernel, in Rust. The honest conclusion: MMQ stays hand-written behind benchmark gates, held by an intrinsic island.

1 The target: the prefill gap is an MMQ gap

Decode (generating one token at a time) is memory-bound and, on this stack, at parity with the hand-tuned baseline on every backend. Prefill (processing a prompt) is compute-bound, and there the DSL-composed engine trails llama.cpp by a consistent margin (dense 0.6–0.9×). That gap has a single name: tensor-core MMQ. llama.cpp quantizes the activation to q8.1 (int8) and contracts block-quantized weights (Q8_0, Q4_K) against it on the hardware *matrix cores* — the units that do a small matrix multiply as one instruction. Closing prefill means expressing that kernel well. The question this paper answers is whether it can be expressed *once*, in the DSL, and lowered to the matrix cores of every backend — the same collapse the rest of the series achieves for other ops.

2 The obstruction: an opaque accumulator

The DSL exposes two routes to the int8 matrix cores, both registered by the lowering engine on NVIDIA:

- **High-level WMMA** (`mma::Matrix`, i8→i32 at 16×16×16): fragments are loaded and stored automatically, one tile at a time. Ergonomic and portable — but the fragment layout is *opaque* to the programmer.

- **Manual MMA** (`mma::MmaDefinition`, `mma.sync` at $16 \times 8 \times 32$): the fragment element-to-lane map *is* exposed (`position_of_nth`), so values can be manipulated in register.

Now the arithmetic MMQ actually needs. For a Q8_0 weight against a q8_1 activation, the contraction is

$$\text{out}[m, n] = \sum_{32\text{-blocks } kb} (\mathbf{xs}[m, kb] \cdot \mathbf{wd}[n, kb]) \cdot \sum_{k \in kb} \mathbf{xq}[m, k] \cdot \mathbf{wq}[n, k],$$

where the inner `int8` dot is the tensor-core op and the per-block `f32` scale $\mathbf{xs} \cdot \mathbf{wd}$ is applied *once per 32-element block*. This is not a kernel choice: `int8` accumulation is only valid *within* one constant-scale block, so the scale must be applied at every block boundary, inherent to the format.

Here is the collision. With the high-level WMMA API the accumulator fragment is opaque, so the only place to apply the per-block scale is to `mma::store` the integer accumulator to shared memory, synchronize, reload it as `f32`, and scale — **once per 32-block**. That store→sync→reload is a full block-wide serialization inserted into the inner loop. It is the wall.

3 The proof: better tiling is slower

A cap could be an artifact of a naïve kernel structure — too few warps, poor operand reuse — rather than the API. To separate “API limit” from “naïve structure,” we built the *fair* ceiling: a 32×64 output tile per block, 8 warps, with A and B staged to shared memory once per 32-block and reused across the tile (A across four N-subtiles, B across two M-subtiles). Occupancy and operand reuse are now fixed; the *only* remaining WMMA-API tax is exactly the per-block scale round-trip through the opaque fragment (Listing 1).

Listing 1: The fair high-level-WMMA ceiling (`mmq_q8_wmma_blk`). A 32×64 tile, 8 warps, operands staged once per block and reused. Everything that would help a naïve kernel is already done; the residual cost is the opaque-fragment scale round-trip. Note the target gate: the DSL’s `int8` matrix-core intrinsic is CUDA-only.

```
#[kernel(targets(cuda), unchecked)] // <-- CUDA-ONLY: int8 MMA is not portable
pub fn mmq_q8_wmma_blk(xq: &Array<i8>, xs: &Array<f32>, wq: &Array<i8>, wd: &Array<f32>,
    out: &mut Array<f32>, #[comptime] m: usize, #[comptime] n: usize,
    #[comptime] k: usize) {
    let (warp, lane) = (UNIT_POS as usize / 32, UNIT_POS as usize % 32); // 8 warps, 2x4 subtiles
    // ... stage A,B to shared once per 32-block, reuse across the tile ...
    // ... per block: mma::execute (i8->i32), then store->sync->reload->scale (the wall) ...
}
```

The measurement is the result. On GB10/Blackwell, the high-level WMMA MMQ reaches only **~7.6 TFLOP/s**, roughly **one-tenth** of the hand-tuned kernel — and the better-tiled version above is *slower* than the naïve one.¹ That is the diagnostic signature of a *sync-bound* kernel: adding tiling adds output per block, which adds scale-round-trips, which adds synchronization — so “more efficient tiling” buys more of the exact cost that dominates. If the kernel were occupancy- or compute-bound, better tiling would help; it hurts, which proves the bottleneck is the serialization the opaque fragment forces.

¹The structural argument (opaque fragment $\Rightarrow$ per-block shared-memory scale round-trip; manual MMA with an exposed lane map as the higher-ceiling path) is documented in the committed `mmq.rs` probe kernels. The TFLOP/s figure and the tiled-is-slower comparison are session measurements on spark (GB10); the probe commit records CUDA verify/bench as pending, so we report them as measured, not as a committed test.

4 The viable route is per-backend anyway

The higher-ceiling path is manual MMA (`mma.sync`), where the fragment element-to-lane map is exposed, so the block scale can be applied *in register* with no shared-memory round-trip — exactly the serialization removed. But that path is per-backend by nature: the DSL’s int8 matrix-core intrinsic is CUDA-only (the probe kernels are literally `#[kernel(targets(cuda))]`, as in Listing 1). Writing it therefore does *not* collapse the product; it produces one more backend-specific kernel, just authored in Rust instead of CUDA C++. The portable-API collapse and the high-ceiling implementation are, for this one operation, mutually exclusive.

5 The honest conclusion, and the seam that holds it

Block-quantized MMQ stays hand-written behind benchmark gates. This is a boundary of the “one source, every backend” program, not a refutation of it. The decode matvecs, the norms, RoPE, attention, GDN, and the entire elementwise fabric — the bulk of the hand-written kernel ledger and the whole of the divergence-bug surface ([7], [5]) — do collapse. The one op that resists is exactly the one the series already built a seam for: an *intrinsic island* ([4]) holds the hand-written tensor-core arm behind a normative, CPU-checkable `default`, so even the un-collapsed kernel keeps one signature, one oracle, one test. A negative result that names its boundary precisely — *which* op, *why* (the opaque fragment), *how you know* (better tiling is slower), and *what contains it* (the island) — is worth more than a portability claim that quietly excludes the hardest kernel. We report it in full so the boundary is known and the seam is used.

References

- [1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL*. Paper 07 (umbrella).
- [2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning*. Paper 07.2.
- [4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch*. Paper 07.3.
- [5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4.
- [6] Hanzo AI. *When the Product Won’t Collapse: An int8 Tensor-Core MMQ Negative Result*. Paper 07.5 (this paper).
- [7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product*. Paper 07.6.